



Gobierno de
México

Ciencia y Tecnología

Secretaría de Ciencia, Humanidades, Tecnología e Innovación



UNRC

Universidad Nacional Rosendo Castellanos
Baja California

Licenciatura en Tecnologías de la Información y la Comunicación

Estructura de Datos Segundo Semestre

Módulo 4. Ordenamiento, búsqueda y análisis de algoritmos

Docente: Lic. Cristina Martínez Ruiz

Alumna: Viviana Selene Reyes Ciénega
Matricula: 25123768-1
Grupo: 208
11 de junio 2026



Introducción

En el ámbito de las Tecnologías de la Información y la Comunicación (TIC), el desarrollo de una solución tecnológica trasciende la simple escritura de instrucciones; exige una administración inteligente y eficiente de los recursos físicos de la computadora, como el procesador y la memoria. Derivado del problema prototípico del programa PAD-Bienestar, existe la necesidad operativa de administrar correctamente el inventario de los 12 equipos destinados a brindar conectividad a la ciudadanía. Un inventario desordenado provoca búsquedas ineficientes que saturan el sistema, de la misma forma que un almacén físico sin catálogo retrasa cualquier operación logística.

El presente reporte técnico documenta el diseño y la implementación de un sistema de gestión desarrollado en el lenguaje C++ para resolver esta problemática. A través de la aplicación de algoritmos clásicos, el programa permite estructurar el inventario mediante el método de ordenamiento de Burbuja y localizar recursos específicos utilizando el método de Búsqueda Binaria.

El objetivo de esta práctica es demostrar cómo la preparación y estructuración previa de los datos —pasando de un estado de caos a uno de orden— reduce drásticamente el número de operaciones que la computadora debe realizar. Asimismo, el sistema integra validaciones de seguridad para evitar la corrupción de la memoria por errores de captura, garantizando así un rendimiento óptimo, seguro y escalable para las operaciones del PAD-Bienestar.

Descripción del Programa

El sistema desarrollado en C++ es una herramienta de gestión para el inventario de equipos tecnológicos del PAD-Bienestar. Su objetivo general es controlar 12 registros de equipos, permitiendo visualizarlos, organizarlos mediante un algoritmo de ordenamiento y localizarlos velozmente a través de un algoritmo de búsqueda. Además, el programa garantiza la integridad de los datos implementando la técnica de persistencia en memoria, lo que permite conservar el arreglo original (datos en bruto) intacto, realizando todas las modificaciones de ordenamiento sobre una copia paralela.

Desarrollo

Enlace al repositorio GitHub: https://github.com/artemusicayvideo-gif/Estructura_de_datos_Modulo-4

Funcionamiento del programa (Lógica algorítmica y Enfoque TIC)

El comportamiento del sistema se rige bajo principios de eficiencia computacional y prevención de errores, desglosados en los siguientes puntos:



- **Copia y Resguardo (Persistencia):**

Al arrancar, el sistema hace un respaldo del arreglo original. En la gestión de Tecnologías de la Información (TIC), esto es el equivalente a respaldar el material original de una producción antes de pasarlo al equipo de edición. Esto asegura que siempre haya un punto de retorno histórico.

```
// Arreglo original: Es el material en bruto (desordenado)
int equiposOriginales[N] = {7, 2, 11, 4, 1, 9, 12, 5, 3, 10, 6, 8};

// Arreglo de trabajo: Es la copia sobre la que vamos a editar y ordenar
int equiposOrdenados[N];

// Hacemos el respaldo inmediatamente al arrancar el programa
copiarArreglo(equiposOriginales, equiposOrdenados);

// Función para copiar los datos (Como respaldar el material en bruto antes de editar)
void copiarArreglo(int origen[], int destino[]) {
    for (int i = 0; i < N; i++) {
        destino[i] = origen[i];
    }
}
```

- **Organización con Método Burbuja:**

Al seleccionar la opción 2, se ejecuta un algoritmo que pertenece a la clase de complejidad cuadrática $O(n^2)$. El programa compara los elementos del arreglo en pares consecutivos (mediante dos ciclos anidados) y los intercambia si están en el orden incorrecto. Este proceso se repite hasta que el conjunto de datos queda estructurado de menor a mayor.

```
case 2:
    ordenamientoBurbuja(equiposOrdenados);
    estaOrdenado = true;
    break;

// El ciclo externo controla cuántas "tomas" o pasadas daremos
for (int i = 0; i < N - 1; i++) {
    // El ciclo interno compara elementos consecutivos
    for (int j = 0; j < N - i - 1; j++) {
        // Si el actual es mayor que el siguiente, se intercambian
        if (equipos[j] > equipos[j + 1]) {
            auxiliar = equipos[j];
            equipos[j] = equipos[j + 1];
            equipos[j + 1] = auxiliar;
        }
    }
}

cout << "\n✅ ¡Organización exitosa! Los equipos fueron ordenados correctamente con el método Burbuja.\n";
```



- **Localización con Búsqueda Binaria:** Una vez ordenado el sistema, el usuario puede buscar equipos (Opción 4). A diferencia de una búsqueda secuencial que revisa uno por uno, la Búsqueda Binaria divide el arreglo a la mitad. Si el valor central no es el buscado, el sistema descarta automáticamente la mitad de los datos donde lógicamente es imposible que se encuentre. Este método tiene una complejidad logarítmica $O(\log n)$, lo cual gestiona de forma altamente eficiente los recursos físicos (procesador) de la computadora.

```
case 4:
    if (!estaOrdenado) {
        cout << "\n❌ Error lógico: Debes ordenar los equipos (opcion 2) antes de poder usar la Busqueda Binaria.\n";
        break;
    }

    // Ejecutamos la búsqueda
    posicion = busquedaBinaria(equiposOrdenados, numeroBuscado);

    while (inicio <= fin) {
        int mitad = (inicio + fin) / 2;

        if (equipos[mitad] == numeroBuscado) {
            return mitad; // Encontramos el equipo
        } else if (numeroBuscado < equipos[mitad]) {
            fin = mitad - 1; // Descartamos la mitad superior
        } else {
            inicio = mitad + 1; // Descartamos la mitad inferior
        }
    }
}
```

- **Blindaje y Validación de Datos:** Se reforzó el código integrando los comandos `cin.clear()` y `cin.ignore()`. Si el operador teclea una letra o un símbolo en lugar de un número válido, estos métodos evitan que la memoria temporal del sistema se corrompa, neutralizando lo que de otra manera causaría un ciclo infinito y la caída (*crash*) del programa.

```
// 🛡 BLINDAJE 1: Validación del Menú para evitar ingreso de letras o símbolos
if (!(cin >> opcion)) {
    cin.clear(); // Limpiamos la alerta de error
    cin.ignore(10000, '\n'); // Expulsamos el carácter inválido de la memoria temporal
    opcion = 0; // Forzamos a que pase al "default" del menú
}

// 🛡 BLINDAJE 2: Validación de la Búsqueda para evitar texto
if (!(cin >> numeroBuscado)) {
    cin.clear();
    cin.ignore(10000, '\n');
    cout << "\n❌ Error de entrada: Solo se permiten caracteres numericos.\n";
    break;
}
```



Evidencia de Ejecución

A continuación, se presentan las pruebas de funcionamiento del sistema PAD-Bienestar, demostrando tanto el flujo ideal de operación como la robustez del programa ante posibles errores de captura por parte de los operadores.

Menú Interactivo Principal

```
===== PAD-BIENESTAR / GESTION DE INVENTARIO =====
1. Mostrar equipos (Orden original de llegada)
2. Ordenar equipos (Metodo Burbuja)
3. Mostrar equipos (Ordenados)
4. Buscar equipo especifico (Busqueda Binaria)
5. Salir del sistema
Selecciona una opcion de produccion: █
```

Registro de Datos (correctos): Mostrar, ordenar y buscar equipo.

```
===== PAD-BIENESTAR / GESTION DE INVENTARIO =====
1. Mostrar equipos (Orden original de llegada)
2. Ordenar equipos (Metodo Burbuja)
3. Mostrar equipos (Ordenados)
4. Buscar equipo especifico (Busqueda Binaria)
5. Salir del sistema
Selecciona una opcion de produccion: 1

█ Inventario de Equipos PAD-Bienestar:
[ 7 2 11 4 1 9 12 5 3 10 6 8 ]

===== PAD-BIENESTAR / GESTION DE INVENTARIO =====
1. Mostrar equipos (Orden original de llegada)
2. Ordenar equipos (Metodo Burbuja)
3. Mostrar equipos (Ordenados)
4. Buscar equipo especifico (Busqueda Binaria)
5. Salir del sistema
Selecciona una opcion de produccion: 2

✓ ¡Organizacion exitosa! Los equipos fueron ordenados correctamente con el metodo Burbuja.

===== PAD-BIENESTAR / GESTION DE INVENTARIO =====
1. Mostrar equipos (Orden original de llegada)
2. Ordenar equipos (Metodo Burbuja)
3. Mostrar equipos (Ordenados)
4. Buscar equipo especifico (Busqueda Binaria)
5. Salir del sistema
Selecciona una opcion de produccion: 3

█ Inventario de Equipos PAD-Bienestar:
[ 1 2 3 4 5 6 7 8 9 10 11 12 ]
```

```
===== PAD-BIENESTAR / GESTION DE INVENTARIO =====
1. Mostrar equipos (Orden original de llegada)
2. Ordenar equipos (Metodo Burbuja)
3. Mostrar equipos (Ordenados)
4. Buscar equipo especifico (Busqueda Binaria)
5. Salir del sistema
Selecciona una opcion de produccion: 4

Ingresa el ID del equipo a buscar (1-12): 4

🔍 ¡Equipo localizado!
ID de equipo: 4
Casillero (Indice) dentro del inventario ordenado: 3

===== PAD-BIENESTAR / GESTION DE INVENTARIO =====
1. Mostrar equipos (Orden original de llegada)
2. Ordenar equipos (Metodo Burbuja)
3. Mostrar equipos (Ordenados)
4. Buscar equipo especifico (Busqueda Binaria)
5. Salir del sistema
Selecciona una opcion de produccion: 5

Cerrando el sistema... Turno finalizado.
selenereyes@MacBook-Air-de-Selene ~ % █
```

Registro de Datos (erroneos): Comportamiento del programa al buscar equipo, cuando aún no se han ordenado.

```
===== PAD-BIENESTAR / GESTION DE INVENTARIO =====
1. Mostrar equipos (Orden original de llegada)
2. Ordenar equipos (Metodo Burbuja)
3. Mostrar equipos (Ordenados)
4. Buscar equipo especifico (Busqueda Binaria)
5. Salir del sistema
Selecciona una opcion de produccion: 4

✗ Error lógico: Debes ordenar los equipos (opcion 2) antes de poder usar la Busqueda Binaria.
```




Manejo de excepciones en la captura de datos: Para garantizar la estabilidad del sistema, se integró un protocolo de validación en el menú principal. Mediante los comandos `cin.clear()` y `cin.ignore()`, el programa bloquea el ingreso de caracteres no válidos (evitando que el sistema colapse) y emite una alerta visual al operador.

```
===== PAD-BIENESTAR / GESTION DE INVENTARIO =====
1. Mostrar equipos (Orden original de llegada)
2. Ordenar equipos (Metodo Burbuja)
3. Mostrar equipos (Ordenados)
4. Buscar equipo especifico (Busqueda Binaria)
5. Salir del sistema
Selecciona una opcion de produccion: 4

❌ Error lógico: Debes ordenar los equipos (opcion 2) antes de poder usar la Busqueda Binaria.
```

Conclusiones

El desarrollo de esta Tarea Auténtica demuestra que en la ingeniería de sistemas no basta con almacenar datos, la eficiencia radica en cómo se estructuran. El ordenamiento de la información es el pilar que sostiene a las búsquedas eficientes; intentar buscar información masiva sin un criterio de orden previo saturaría el rendimiento de los componentes físicos. Como se pudo comprobar con la búsqueda binaria, preparar el entorno de trabajo (ordenar el arreglo) permite reducir drásticamente el número de operaciones que la computadora debe realizar. Trasladando esto a un entorno profesional de organización logística —como el de una agencia de publicidad—, estructurar correctamente los recursos desde la preproducción es la única garantía de que el flujo de trabajo será ágil, seguro y libre de colapsos.

Referencias:

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2018). *Algoritmos: Teoría y práctica* (3.^a ed.). MIT Press / McGraw-Hill Interamericana.
- Deitel, P. J., & Deitel, H. M. (2014). *C++ Cómo programar* (9.^a ed.). Pearson Educación.
- Joyanes Aguilar, L. (2017). *Fundamentos de programación: algoritmos, estructuras de datos y objetos* (4.^a ed.). McGraw-Hill.
- Universidad Nacional Rosario Castellanos. (2026). *Módulo 4: Ordenamiento, búsqueda y análisis de algoritmos. Estructura de Datos*. Licenciatura en Tecnologías de la Información y la Comunicación.